

Security Architecture [VERY OUTDATED]

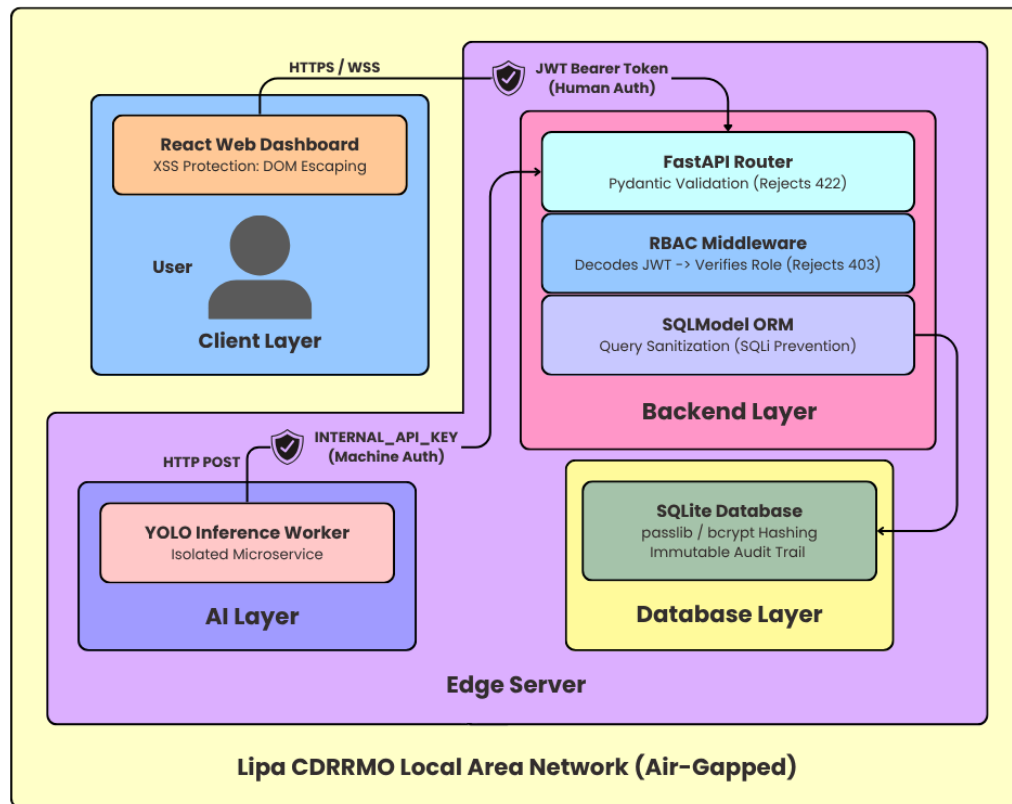


Figure 6.0. Security Architecture Diagram

The system's security framework was designed to safeguard sensitive municipal traffic surveillance data, restrict unauthorized access to the web dashboard, and maintain the forensic integrity of all incident logs. Given the confidentiality of the data and the nature of emergency response operations, security protocols were integrated throughout the network, application, and database layers.

Data Localization and Network Isolation

The system's primary defensive mechanism was its physical deployment strategy. In compliance with NFR-17 (Data Localization), all components were

hosted on an on-premises Edge Inference Server located within the Lipa CDRMO command center. Data were never transmitted to or stored on public cloud infrastructure. This air-gapped approach eliminated external, internet-based threat vectors. Internal data transmissions between client browsers and the edge server were secured using Hypertext Transfer Protocol Secure (HTTPS) for RESTful API queries and WebSocket Secure (WSS) for real-time alert broadcasting.

Dual-Vector Authentication System

The system managed incoming traffic by employing two distinct authentication mechanisms, each tailored for its user type. For human users interacting with the dashboard through the React frontend, stateless, session-based authentication was implemented using JWT. When a user logged in successfully, the FastAPI backend generated a time-limited token with `pyjwt`, set to expire after a standard eight-hour operational shift. This token was required to be included as a Bearer token in the HTTP Authorization header for all subsequent dashboard actions.

In contrast, the AI inference engine, which operated continuously as a background microservice, did not utilize the JWT authentication flow. Instead, the AI worker communicated with the backend's internal webhooks using a statically defined and cryptographically secure internal API key. This key, referred to as `INTERNAL_API_KEY`, was injected through environment variables. By using this method, the system ensured that unauthorized scripts on the local network could not trigger false accident alerts.

Role-Based Access Control

Authorization was governed by an RBAC matrix integrated into the FastAPI dependency injection pipeline. User clearance levels (Administrator or Operator) were encoded within the JWT payload. When a client requested access to a protected route (e.g., modifying a user account), the backend intercepted the request, decoded the JWT, and verified the user's role. If an Operator attempted to access an Administrator-exclusive endpoint, the server rejected the request with a 403 Forbidden error before any internal logic was executed.

Cryptographic Standards and Password Security

Plain-text passwords were never stored in the database or retained in server memory. The system used the passlib library to implement bcrypt, an industry-standard cryptographic hashing algorithm. Bcrypt applied a randomized salt to each password before hashing, neutralizing pre-computed dictionary attacks and rainbow-table vulnerabilities, even in the event of a physical compromise of the local edge server.

Threat Mitigation and Input Validation

The security architecture was designed to neutralize common web application vulnerabilities:

- SQL Injection (SQLi): The backend used an ORM. All database queries were parameterized and sanitized by it before execution, making it impossible for malicious SQL injection via the frontend to be executed by the database engine.
- Cross-Site Scripting (XSS): The React frontend protected against XSS attacks by escaping all dynamic variables and data payloads before

rendering them into the Document Object Model (DOM). Additionally, strict TypeScript interfaces ensured that WebSocket payloads conformed to expected data types.

- Cross-Site Request Forgery (CSRF): The use of JWT Bearer tokens passed explicitly in HTTP headers, rather than automated browser session cookies, rendered the system structurally immune to CSRF attacks. Malicious background tabs could not forge authenticated requests to the edge server.
- Data Validation: All incoming data from the React dashboard and the AI engine underwent strict structural and type validation using Pydantic models prior to processing. This ensured that malformed or incomplete payloads were rejected with a 422 Unprocessable Entity response.

Forensic Integrity and Auditability

To preserve the legal and operational integrity of the system's history, the database prohibited hard deletion of relational records. The User and Camera entities used a boolean `is_active` flag. When an Administrator deleted a user or camera, the system performed a soft delete by toggling the flag to false. This action immediately revoked system access and halted AI polling, while maintaining the referential integrity of historical incident logs. As a result, the CDRRMO retained an immutable audit trail indicating which operator verified or dismissed a specific accident at any given time.